Práctica 3. Paralelismo en sistemas de memoria distribuida (MPI)

Contenidos

1. Objetivos 2
2. Introducción a MPI 2
3. Desarrollo 3
Tarea 0. Aspectos que impactan en un diseño paralelo 3
Entregables de esta tarea 5
Tarea 1. Análisis del rendimiento secuencial mediante `gprof` 6
Entregables de esta tarea 7
Tarea 2. Ejecución distribuida de código MPI 7
Introducción a MPI 7
Tutorial de compilación con MPI 8
Tutoriales acerca de programación con MPI 3
Entregable de esta tarea 8
Tarea 3. Paralelización del código identificado en la práctica anterior 9
Identificación 9
Implementación 9
Evaluación 9
Entregables de esta tarea 10
Tarea 4. Paralelización de un clasificador kNN mediante MPI 11
Contexto del problema 11
Aprendizaje automático (Machine Learning) 11
Evaluación de clasificadores 12
Validación cruzada (Cross-Validation) 12
Clasificador k-Nearest Neighbors (kNN) 13
Ajuste de hiperparámetros *(Fine-Tuning)* 14
Caso especial: paralelismo funcional y ejecución especulativa 14
Pasos recomendados 15
Entregables de esta tarea 19

1. Objetivos

- Aprender a utilizar herramientas de profiling, como `gprof`, para determinar qué funciones suponen un cuello de botella durante la ejecución.
- A partir del análisis mediante dicha herramienta, determinar la parte crítica del código de la práctica anterior (práctica 2) que se podría beneficiar del paralelismo, y aplicarlo.
- Por otro lado, obtener una versión paralela con MPI de la solución secuencial para el problema proporcionado por el profesorado.
- Analizar y explotar en todos los casos los diferentes tipos posible de paralelismo que se perciben en la solución secuencial dada.
- Realizar pruebas suficientes que permitan estimar valores de rendimiento de la versión paralela frente a la secuencial.
- Visualizar los datos de rendimiento de la forma más adecuada para poder argumentar los resultados obtenidos y relacionarlos con las decisiones de paralelización tomadas.

2. Introducción a MPI

MPI (Message-passing interface) es una especificación que establece las funciones de una biblioteca para el paso de mensajes entre múltiples procesadores. Esto permite la comunicación vía paso de mensajes entre nodos de un clúster de computadores que constituyen sistemas multicomputador de memoria distribuida1. En este escenario los datos utilizados por un proceso se mueven desde el espacio de direcciones de este al de otro proceso, de forma cooperativa, mediante las instrucciones recogidas en el estándar MPI.

Podemos distinguir 4 tipos de instrucciones definidas en la especificación de MPI:

1. Instrucciones para abrir, gestionar y cerrar las comunicaciones entre procesos corriendo en diferentes nodos.
2. Instrucciones para transferir datos entre 2 procesos (uno a uno).
3. Instrucciones para transferir datos entre múltiples procesos.
4. Instrucciones que permiten al usuario definir tipos de datos.

Las principales ventajas del uso de MPI son que permite establecer un estándar de paso de mensajes en arquitecturas multicomputadora portable, fácil de utilizar y que permite abstraer los detalles de bajo nivel propios de la gestión de este tipo de comunicaciones.

Existen multitud de librerías que implementan el estándar MPI (mpich, OpenMPI, ...) en diferentes lenguajes como c, c++, Fortran, Python etc.. En este enlace podéis encontrar un tutorial con ejemplos de uso de MPI del laboratorio Lawrence-Livermore.

Tutoriales para la programación con MPI

Podréis encontrar más información acerca de las interfaces de paso de mensajes, y el modelo de programación (tal y como se ha visto en teoría), además de las funciones de comunicación colectiva, en los siguientes vídeos preparados por la Universitat Politècnica de València (UPV):

- https://youtu.be/uiZU05CfZUg?si=oS5gU-AqjUGzRKl3
- https://youtu.be/y1yB7LTn6oA?si=dO7IdLa8QmGMS9CI

O este tutorial de la Universidad de Granada:

- https://lsi2.ugr.es/jmantas/ppr/tutoriales/tutorial_mpi.php
- https://lsi2.ugr.es/jmantas/ppr/tutoriales/tutorial_mpi.php?tuto=03_pi

3. Desarrollo

En este caso, se desarrollará la práctica en múltiples computadores (mínimo 3), y se usarán las estrategias de paralelización en entornos multicomputador que más se adecúen a la estrategia particular que siga cada grupo. Es importante argumentar todos los cambios necesarios para paralelizar la solución secuencial de partida. El análisis del rendimiento debe ser exhaustivo y detallado (diferentes tallas del problema, diferente número de nodos en el clúster...). El informe debe incluir los aspectos estudiados en la Unidad 3 de teoría: modo, estilo y estructura del programa paralelo, características de la comunicación, etc.

El análisis se realizará tanto con el código proporcionado, como con el código secuencial identificado en la práctica anterior, e incluirá al menos los siguientes aspectos (con los gráficos necesarios):

- Análisis del tiempo y número de llamadas a cada función del código secuencial (mediante análisis con herramientas de profiling, como gprof).
- Tiempos y aceleración en términos de nodos y tamaño del problema/umbrales de error
- Eficiencia paralela en términos de los parámetros anteriores → ¿Cuál es la implementación más eficiente?
- Comparación con la implementación secuencial

Tarea 0. Aspectos que impactan en un diseño paralelo

A continuación, se presentan 2 códigos que tratan de ejemplificar aspectos a habituales a tener cuenta a la hora de paralelizar un algoritmo. Su comprensión puede ser de utilidad para las siguientes tareas.

Continúa en la página siguiente ...

El problema de los vectores dinámicos en el paralelismo: Analizad el siguiente código en el cual se inicializa un vector STL de gran tamaño de dos formas diferentes. Ejecutadlo y responded a las preguntas planteadas.

```
#include <iostream>
#include <vector>
#include <time.h>
#include <ctime>

#define SIZE 1000000

using namespace std;

int main()
{
    // A:
    clock_t begin = clock();
    // declaramos un vector sin inicializar.
    std::vector<int> a;
    // usamos push_back para añadir cada elemento.
    for(int i=0;i<SIZE;i++)
    {
        a.push_back(i);
    }
    double aTime = double(clock() - begin) / CLOCKS_PER_SEC;
    cout<<"Elapsed time: "<<aTime<<"s"<<endl;

    // B:
    begin = clock();
    // declaramos un vector inicializado al tamaño requerido.
    std::vector<int> b(SIZE);
    for(int i=0;i<SIZE;i++)
    {
            b[i] = i;
    }
    double bTime = double(clock() - begin) / CLOCKS_PER_SEC;
    cout<<"Elapsed time: "<<bTime<<"s"<<endl;
    cout<<"Gain: "<<aTime/bTime<<endl;
}
```

- ¿Qué diferencias observáis entre ambas versiones?
- ¿Qué versión es más rápida?
- ¿A qué se debe la diferencia de tiempo entre la primera y la segunda versión?
- ¿Se podrían paralelizar ambas versiones? Razonad la respuesta.

Recorrido de matrices: Analizad el siguiente código en el cual se recorre una matriz de gran tamaño de dos formas diferentes. Ejecutadlo y responded a las preguntas planteadas.

```
#include <iostream>
#include <vector>
#include <time.h>
#include <ctime>

const int SIZE = 1000;

using namespace std;

int main()
{
    // A:
    clock_t begin = clock();
    // declarar una matriz de tamaño SIZExSIZE.
    int a[SIZE][SIZE];
    // una matriz in c se almacena en memoria como un array de una única
    // dimensión por filas.
    // Para una matriz de 3x3:
    // 1 2 3
    // 4 5 6
    // 7 8 9
    // se almacena en memoria como el array: 1 2 3 4 5 6 7 8 9
    // si recorremos la matriz por columnas, visitamos las celdas en el siguiente
    // orden: 1 4 7 2 5 8 3 6 9
    for(int i=0;i<SIZE;i++)
    {
        for(int j=0;j<SIZE;j++)
            a[j][i] = i+j;
    }
    double aTime = double(clock() - begin) / CLOCKS_PER_SEC;
    cout<<"Elapsed time: "<<aTime<<"s"<<endl;

    // B:
    begin = clock();
    int b[SIZE][SIZE];
    // si recorremos la matriz por filas, visitamos las celdas en el siguiente
    // orden: 1 2 3 4 5 6 7 8 9
    for(int i=0;i<SIZE;i++)
    {
        for(int j=0;j<SIZE;j++)
            b[i][j] = i+j;
    }
    double bTime = double(clock() - begin) / CLOCKS_PER_SEC;
    cout<<"Elapsed time: "<<bTime<<"s"<<endl;
    cout<<"Gain: "<<aTime/bTime<<endl;
}
```

- ¿Qué diferencias hay entre ambas versiones?
- Probad diferentes tamaños de matriz hasta que observéis diferencias notables de tiempo.
    - ¿Qué versión se ejecuta más rápido?
    - ¿A partir de qué tamaño comienza a notarse la diferencia de tiempo?
    - ¿A qué se debe la diferencia de tiempo entre ambas versiones?

Entregables de esta tarea

A parte de las respuestas a las preguntas anteriores, con las conclusiones de este apartado, ¿qué cambios son necesarios en vuestro código o en el código proporcionado antes de aplicar paralelismo mediante MPI? Explicadlos.

Tarea 1. Análisis del rendimiento secuencial mediante `gprof`

Antes de comenzar la paralelización de cualquier programa, es necesario identificar qué funciones consumen más tiempo de ejecución y, por tanto, cuáles son las principales candidatas a optimización. Para ello, utilizaremos **gprof**, una herramienta de *profiling* de GNU que permite analizar el tiempo de CPU consumido por cada función, el número de veces que se invoca y el tiempo medio por llamada. Además, proporciona un grafo de llamadas (*call graph*) que permite identificar las relaciones entre las distintas funciones del programa.

Para utilizar `gprof`, es necesario compilar y enlazar el programa con la opción `-pg`, que habilita la instrumentación necesaria para registrar las llamadas a funciones. Se recomienda utilizar también `-g` para incluir información de depuración y `-O0` para evitar que las optimizaciones del compilador, especialmente el *inlining*, dificulten la identificación de las funciones:

```
gcc -O0 -g -pg programa.c -o programa
```

En programas C++, se utilizará `g++` en lugar de `gcc`. Si el programa contiene varios archivos fuente, todos deberán compilarse con `-pg`, y esta opción también deberá incluirse durante el enlazado.

A continuación, se ejecuta normalmente el programa:

```
./programa
```

Al finalizar, se generará un archivo denominado `gmon.out`, que contiene la información recogida durante la ejecución. Para analizarla y generar un informe, se utilizará:

```
gprof -b -p -q ./programa gmon.out > perfil.txt
```

Las opciones utilizadas permiten obtener:

- **-b**: elimina las explicaciones generales del informe para facilitar su lectura.
- **-p**: genera el perfil plano (*flat profile*), que incluye el tiempo de CPU atribuido a cada función, su porcentaje respecto al total, el número de llamadas y el tiempo medio por llamada.
- **-q**: genera el grafo de llamadas (*call graph*), que muestra las relaciones entre funciones y el tiempo atribuido a cada una, incluyendo sus llamadas a otras funciones.

A partir de estos resultados, se deberán identificar las funciones que concentran la mayor parte del tiempo de ejecución, distinguiendo entre aquellas que se ejecutan muchas veces y aquellas cuyo coste individual es elevado. Los tiempos proporcionados por gprof son estimaciones basadas en muestreo, mientras que el número de llamadas se obtiene mediante instrumentación. Por tanto, los resultados deben interpretarse como una aproximación al comportamiento del programa y utilizarse para justificar qué partes del código conviene estudiar para su posterior paralelización mediante MPI.

Nota: La opción -O0 anterior, permite evitar el inlining de funciones, pero es conveniente realizar una segunda compilación con -O2, ya que los cuellos de botella pueden cambiar al activar las optimizaciones. Para seleccionar las funciones que realmente interesa paralelizar, el perfil de la versión optimizada es generalmente más representativo. Comparad la salida de ambas opciones, y seleccionad las funciones a optimizar en base a los resultados obtenidos.

Entregables de esta tarea

De esta tarea se espera obtener:

- Los informes generados por gprof tanto para el código propio de la práctica anterior, como para el código proporcionado (en formato visual, no el fichero generado).
- Las conclusiones a las que hayáis llegado, en cuanto a las funciones que son mejores candidatas a la optimización, que deberán constar en la memoria.

Tarea 2. Ejecución distribuida de código MPI

Introducción a MPI

Antes de emprender la paralelización de los problemas propuestos, es necesario familiarizarse con el funcionamiento de MPI, para ello, ejecutaremos un código de ejemplo sencillo, a modo de 'hola mundo' paralelo. Este es el código a compilar:

```
#include <stdio.h>
#include "mpi.h"
#include <unistd.h>
#include <stdlib.h>

int sched_getcpu();

int main(int argc, char *argv[]) {
  int numprocs, rank, namelen;
  char processor_name[MPI_MAX_PROCESSOR_NAME];

  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Get_processor_name(processor_name, &namelen);
  printf(" PID: %d Hello from process %d out of %d on %s processor %d \n",
         getppid(), rank, numprocs, processor_name, sched_getcpu());
  if (rank==2)
  {
     printf("¡Hola! desde el procesador %d\n", rank);
  }
  MPI_Finalize();
}
```

Tutorial de compilación con MPI

1. Todas las máquinas deben tener el **mismo usuario**. Todos los ejecutables deben tener el **mismo nombre** en cada máquina y estar en la **misma carpeta**.
2. Compilad empleando `mpicc` o `mpic++`:

```
mpicc programa.c -o programa
```

Recordad que el programa, junto con sus dependencias, deben copiarse y ser accesibles en cada nodo de su multicomputador.

3. Cread el **archivo de hosts** con las IPs o nombres de cada nodo de la red y copiar al resto de máquinas en el mismo directorio de trabajo donde se encuentran los ejecutables.

Archivo de hosts: archivo de texto plano que especifica el nombre o la IP de las máquinas que forman el supercomputador. Cada nombre/IP en una línea.

4. Cree un certificado de clave pública SSH en uno de los nodos:

```
ssh-keygen -t rsa
```

Cread la carpeta oculta `~/.ssh`.

5. Copiad el archivo `~/.ssh/id_rsa.pub` en la carpeta `~/.ssh` del resto de máquinas y cambiad su nombre a `authorized_keys`. Si no existe la carpeta oculta `~/.ssh` en una máquina, creadla previamente ejecutando el comando:

```
ssh-keygen -t rsa
```

6. Ejecutad el programa en la máquina donde se creó el certificado:

```
mpirun -mca plm_rsh_no_tree_spawn 1 -hostfile <archivo_hosts> -n <núm_procesos> ./programa
```

Observaciones

- Usad el mismo directorio en todas las máquinas: p. ej., carpeta personal (`$HOME`).
- Abrid sesión con el **mismo usuario en todos los nodos** del clúster.
- Puede ser necesario modificar los permisos de los archivos:

Archivo de hosts:

```
chmod 600 <archivo_hosts>
```

Archivo ejecutable para permitir ejecución remota:

```
chmod o+x programa
```

Entregable de esta tarea

Deberéis mostrar al profesorado de prácticas que habéis sido capaces de lanzar el ejemplo en varios ordenadores de la sala de prácticas.

Tarea 3. Paralelización del código identificado en la práctica anterior

Identificación

Tras la ejecución de la herramienta de profiling (gprof), así como con los resultados de la práctica anterior (grafo de dependencias, etc.) identificad cuál es la función o funciones (o parte crítica del código) que mayor sentido tiene paralelizar, es decir, la que aportará mayor ganancia si se paraleliza basándonos en el número de veces que se ejecuta, y el tiempo que tarda.

Implementación

Utilizando las técnicas de paralelismo mediante paso de mensajes utilizando MPI, implementad una versión paralela del código secuencial propuesto anteriormente.

Evaluación

Comentad los fragmentos más relevantes de las versiones secuencial y paralela, prestando especial atención a:

- Los parámetros que determinan el tamaño del problema.
- Las estructuras de control que desempeñan un papel importante en el algoritmo.
- Las modificaciones realizadas para transformar la versión secuencial en una solución paralela, justificando las decisiones adoptadas.
- Las funciones MPI utilizadas, explicando su finalidad y su papel en la distribución del trabajo, la comunicación y la sincronización entre procesos.

Responded de forma razonada a las siguientes cuestiones:

1. Calculad la ganancia en velocidad (speed-up) en función del número de procesos MPI y de los parámetros que determinan el tamaño del problema. Representad los resultados mediante gráficas que faciliten su interpretación.
2. Analizad cómo evoluciona el rendimiento al aumentar el número de procesos. ¿Se obtiene siempre una mejora? ¿Qué factores limitan la escalabilidad de la implementación?
3. Estudiad el impacto de las comunicaciones entre procesos. ¿Qué proporción del tiempo de ejecución se dedica al procesamiento y cuál a las comunicaciones y la sincronización? ¿Cómo influye este reparto en el rendimiento?
4. Comparad las distintas estrategias de paralelización implementadas. ¿Qué estrategia proporciona mejores resultados y en qué condiciones? Justificad vuestra respuesta.
5. ¿Cuál sería la ganancia máxima teórica? ¿Qué factores impiden alcanzarla en vuestra implementación?

6. Comparad la eficiencia de las implementaciones secuencial y paralela. ¿Cuál obtiene mejores resultados para los distintos tamaños del problema y configuraciones estudiadas?

Entregables de esta tarea

La versión paralela del código, así como las respuestas a las cuestiones de 'Evaluación', que deberán constar en vuestra memoria.

Tarea 4. Paralelización de un clasificador kNN mediante MPI

En esta tarea, todos los grupos deberéis paralelizar un programa secuencial proporcionado por el profesorado. Para ello, comenzaréis analizando su funcionamiento e identificando las partes susceptibles de ejecutarse en paralelo. Posteriormente, utilizaréis MPI (Message Passing Interface) para desarrollar una versión paralela que distribuya el trabajo entre diferentes procesos.

El objetivo es aplicar los conceptos de programación paralela mediante paso de mensajes, prestando especial atención a la distribución de datos y tareas, las dependencias entre operaciones, los mecanismos de comunicación y la sincronización entre procesos.

Contexto del problema

El objetivo de esta práctica es paralelizar un programa que realiza el entrenamiento, ajuste de hiperparámetros y evaluación de un clasificador k-Nearest Neighbors (kNN) mediante técnicas de validación cruzada (cross-validation) y búsqueda de hiperparámetros (fine-tuning).

Para ello, utilizaremos el conjunto de datos MNIST, ampliamente empleado en tareas de reconocimiento de caracteres mediante técnicas de visión por computador. Este conjunto contiene imágenes de dígitos manuscritos (del 0 al 9) y está formado por 60.000 muestras de entrenamiento y 10.000 muestras de prueba.

Aprendizaje automático (Machine Learning)

El aprendizaje automático es una rama de la inteligencia artificial cuyo objetivo es desarrollar algoritmos capaces de aprender patrones a partir de datos y utilizarlos para realizar tareas sin necesidad de programar explícitamente todas las reglas que deben seguir.

Una de sus aplicaciones más habituales es la clasificación, que consiste en asignar cada muestra de entrada a una de las categorías previamente definidas.

Según la información disponible durante el aprendizaje, podemos distinguir tres enfoques principales:

1. **Aprendizaje supervisado:** el algoritmo utiliza un conjunto de muestras previamente etiquetadas. Su objetivo es aprender las características que permiten distinguir unas categorías de otras y utilizar ese conocimiento para clasificar nuevas muestras.
2. **Aprendizaje no supervisado:** el algoritmo trabaja con muestras que no disponen de etiquetas y trata de identificar patrones, similitudes o estructuras presentes en los datos. Por ejemplo, puede agrupar muestras con características semejantes.
3. **Aprendizaje semisupervisado:** combina muestras etiquetadas y no etiquetadas durante el proceso de aprendizaje, aprovechando la información de ambos conjuntos.

Evaluación de clasificadores

Para evaluar el rendimiento de un clasificador existen diferentes métricas, entre las que destacan la exactitud (*accuracy*), la precisión (*precision*) y la sensibilidad (*recall*). Estas métricas permiten cuantificar hasta qué punto el clasificador asigna correctamente las categorías a las muestras analizadas.

Sin embargo, obtener buenos resultados sobre los datos utilizados durante el entrenamiento no garantiza que el clasificador funcione correctamente con muestras nuevas. Uno de los problemas más habituales es el sobreajuste (*overfitting*), que se produce cuando un modelo se adapta excesivamente a los datos de entrenamiento y pierde capacidad de generalización.

Para evaluar correctamente un clasificador, el conjunto de datos suele dividirse en dos subconjuntos:

- **Conjunto de entrenamiento (training set):** se utiliza para construir o ajustar el clasificador.
- **Conjunto de prueba (test set):** se utiliza para evaluar su rendimiento final con muestras que no han participado en el proceso de entrenamiento o ajuste.

Además, durante el desarrollo del clasificador es habitual reservar una parte de los datos de entrenamiento como conjunto de validación (*validation set*). Este conjunto permite comparar distintas configuraciones del clasificador y seleccionar los valores de sus hiperparámetros sin utilizar los datos de prueba.

Validación cruzada (Cross-Validation)

La validación cruzada es una técnica que permite evaluar de forma más robusta el rendimiento de un clasificador utilizando diferentes particiones de los datos disponibles.

En lugar de realizar una única división entre entrenamiento y validación, se repite el proceso utilizando distintos subconjuntos. El rendimiento final se estima combinando los resultados obtenidos en las distintas evaluaciones, habitualmente mediante su media.

Entre las técnicas de validación cruzada destacan las siguientes:

1. **Validación cruzada de N particiones (N-Fold Cross-Validation):** el conjunto de datos se divide en N subconjuntos, denominados folds. Se realizan N evaluaciones y, en cada una de ellas, se utiliza un subconjunto diferente como conjunto de validación, mientras que los N−1 restantes se emplean para el entrenamiento. De este modo, cada muestra participa una vez en la validación.
2. **Validación dejando uno fuera (Leave-One-Out Cross-Validation):** es un caso particular de la validación N-Fold en el que N coincide con el número de muestras disponibles. En cada evaluación se utiliza una única muestra para validar el clasificador

y todas las restantes para entrenarlo. Aunque permite aprovechar prácticamente todos los datos en cada evaluación, su coste computacional puede resultar muy elevado.

Clasificador k-Nearest Neighbors (kNN)

El clasificador k-Nearest Neighbors (kNN) es uno de los algoritmos de clasificación supervisada más sencillos.

Su funcionamiento se basa en una idea intuitiva: las muestras que presentan características similares suelen pertenecer a la misma categoría. Por tanto, para clasificar una nueva muestra, el algoritmo busca las K muestras más próximas a ella dentro del conjunto de entrenamiento y determina su categoría a partir de las etiquetas de esos vecinos.

El procedimiento de clasificación consta de los siguientes pasos:

1. Calcular la distancia entre la muestra que queremos clasificar y cada una de las muestras del conjunto de entrenamiento.
2. Identificar las K muestras que se encuentran a menor distancia.
3. Determinar la categoría más frecuente entre esos K vecinos.
4. Asignar dicha categoría a la nueva muestra.

El valor de K es un hiperparámetro fundamental del clasificador. Un valor pequeño puede hacer que el resultado sea muy sensible al ruido o a muestras atípicas, mientras que un valor elevado puede provocar que se pierdan diferencias relevantes entre categorías.

A diferencia de otros algoritmos de aprendizaje automático, kNN no necesita construir un modelo explícito durante una fase de entrenamiento. En su forma más sencilla, el entrenamiento consiste únicamente en almacenar las muestras etiquetadas que posteriormente se utilizarán para realizar las clasificaciones.

Esta característica facilita la incorporación de nuevas muestras e incluso de nuevas categorías sin necesidad de volver a entrenar un modelo. Por ejemplo, en una aplicación de reconocimiento facial, podrían incorporarse nuevos usuarios añadiendo imágenes etiquetadas de sus rostros al conjunto de entrenamiento.

Sin embargo, esta sencillez tiene una contrapartida importante: **el coste computacional se concentra en la fase de clasificación**, ya que, para cada nueva muestra, es necesario calcular las distancias respecto a las muestras almacenadas.

Este comportamiento convierte a kNN en un candidato interesante para estudiar distintas estrategias de paralelización, tanto mediante la distribución de muestras entre procesos como mediante la descomposición de las operaciones necesarias para clasificarlas.

Ajuste de hiperparámetros *(Fine-Tuning)*

Los clasificadores suelen disponer de hiperparámetros que permiten adaptar su funcionamiento a las características de cada problema.

En la implementación de kNN que utilizaremos en esta práctica, consideraremos un único hiperparámetro: **el número de vecinos K.**

El objetivo del ajuste de hiperparámetros es encontrar el valor de K que proporciona el mejor rendimiento de clasificación.

Una estrategia habitual consiste en realizar una búsqueda exhaustiva (grid search), evaluando sistemáticamente diferentes valores de los hiperparámetros mediante validación cruzada.

En nuestro caso, el procedimiento será el siguiente:

1. Seleccionar un conjunto de posibles valores de K.
2. Realizar una validación cruzada para cada valor de K.
3. Obtener una estimación del rendimiento del clasificador para cada configuración.
4. Comparar los resultados y seleccionar el valor de K que proporciona el mejor rendimiento.

El resultado de este proceso será un vector que contendrá una medida de rendimiento para cada valor de K evaluado.

Si un clasificador tuviera varios hiperparámetros, la búsqueda exhaustiva evaluaría diferentes combinaciones de sus valores, dando lugar a una estructura de resultados multidimensional.

Caso especial: paralelismo funcional y ejecución especulativa

Además del paralelismo de datos que puede identificarse en diferentes partes del algoritmo, esta práctica plantea un caso especial cuyo objetivo es explorar el paralelismo funcional, la comunicación entre procesos y la ejecución especulativa.

En una implementación convencional, las fases de ajuste de hiperparámetros y evaluación final del clasificador se ejecutan secuencialmente:

1. Se evalúan distintos valores de K mediante validación cruzada.
2. Se selecciona el valor de K que proporciona el mejor rendimiento.
3. Se utiliza ese valor para realizar la evaluación final del clasificador sobre el conjunto de prueba.

Existe, por tanto, una dependencia entre ambas fases: no podemos realizar la evaluación final con el valor óptimo de K hasta que haya terminado el proceso de ajuste.

Para explorar una estrategia alternativa, vamos a forzar la ejecución concurrente de ambas fases mediante procesos MPI, utilizando un mecanismo de ejecución especulativa.

El funcionamiento propuesto será el siguiente:

1. Un conjunto de procesos se encargará de realizar el ajuste de hiperparámetros, evaluando diferentes valores de K.
2. Simultáneamente, otro conjunto de procesos comenzará a realizar la evaluación final utilizando el mejor valor de K conocido hasta ese momento.
3. Cuando el proceso de ajuste encuentre un nuevo valor de K que mejore los resultados anteriores, deberá comunicarlo a los procesos responsables de la evaluación mediante paso de mensajes.
4. Si la evaluación está utilizando un valor de K que ha dejado de ser el mejor conocido, deberá invalidar el procesamiento en curso y reiniciarlo con el nuevo valor.
5. Cuando termine el ajuste de hiperparámetros, la evaluación final deberá haberse realizado correctamente utilizando el valor óptimo de K encontrado.

Esta estrategia es especulativa porque la evaluación final comienza utilizando un valor de K que todavía no sabemos si será el definitivo.

Si el valor utilizado resulta ser el óptimo, habremos conseguido solapar parte de las dos fases y reducir el tiempo total de ejecución. Por el contrario, si el mejor valor de K cambia repetidamente, será necesario descartar trabajo ya realizado, lo que puede reducir o incluso eliminar las ventajas de la paralelización.

El objetivo será analizar si el paralelismo funcional puede proporcionar una mejora de rendimiento suficiente para compensar los costes adicionales de comunicación, sincronización y procesamiento especulativo.

La implementación deberá tener en cuenta que los procesos MPI disponen de espacios de memoria independientes. Por tanto, cualquier información que necesite compartirse entre procesos, como los resultados parciales de validación, los mejores valores de K o las señales de finalización, deberá intercambiarse mediante mecanismos de comunicación explícitos.

Pasos recomendados

a. Preparación del entorno y ejecución de la versión secuencial

Antes de comenzar, leed atentamente el archivo README incluido junto con el código fuente de la práctica y seguid las instrucciones necesarias para instalar las dependencias y compilar el programa.

En particular, deberéis disponer de OpenCV y de las herramientas de compilación necesarias.

Para descargar el conjunto de datos, ejecutad:

```
chmod a+x get_dataset.sh
./get_dataset.sh
```

Para compilar el programa secuencial:

```
cd build
make
```

Para ejecutar la versión secuencial:

```
./cv
```

Para desarrollar la versión paralela, deberéis disponer de una implementación de MPI, como Open MPI, y de sus herramientas de compilación y ejecución.

La compilación de un programa MPI escrito en C++ se realiza habitualmente utilizando mpic++. Por ejemplo:

```
mpic++ -O2 -o knn_mpi knn_mpi.cpp
```

Y su ejecución con cuatro procesos:

```
mpirun -np 4 ./knn_mpi
```

Estos comandos son orientativos y deberán adaptarse a la estructura del proyecto y sus dependencias.

b. Análisis de la solución secuencial

Analizad el código fuente proporcionado hasta comprender su funcionamiento general y las operaciones que realiza cada una de sus funciones.

Prestad especial atención a los aspectos que pueden influir en el diseño de una solución paralela, como las variables utilizadas, el tamaño del problema, las dependencias entre operaciones, las estructuras de control y los datos que será necesario intercambiar entre procesos.

Identificad las funciones que concentran la mayor parte del tiempo de ejecución. Para ello, utilizad las herramientas de análisis de rendimiento (profiling) estudiadas en las tareas anteriores.

c. Representación de la solución secuencial

Elaborad un grafo de control de flujo que represente el funcionamiento de la solución secuencial e identifique sus principales fases de procesamiento. Os podréis ayudar de la salida de `gprof` de la tarea 1.

Indicad qué partes del algoritmo presentan dependencias y cuáles podrían ejecutarse concurrentemente.

d. Diseño e implementación de la solución paralela

Utilizando MPI y teniendo en cuenta el análisis anterior, diseñad e implementad una versión paralela del programa.

Tratad de identificar y aprovechar los diferentes tipos de paralelismo presentes en el problema, prestando especial atención a los siguientes aspectos:

- La distribución del trabajo y de los datos entre los distintos procesos.
- La identificación de tareas independientes que puedan ejecutarse concurrentemente.
- Los mecanismos de comunicación necesarios para intercambiar datos y resultados.
- Las dependencias entre operaciones y los puntos en los que sea necesaria la sincronización.
- El equilibrio de carga entre procesos.
- Los costes de comunicación y su influencia en el rendimiento global.

Considerad diferentes estrategias de distribución del trabajo y justificad las decisiones adoptadas.

e. Evaluación del rendimiento

Realizad pruebas de ejecución con diferentes configuraciones y tamaños del problema.

Medid los tiempos de ejecución de las versiones secuencial y paralela, y calculad las ganancias de rendimiento obtenidas.

Para medir el tiempo de ejecución de la versión paralela, podéis utilizar MPI_Wtime(), que proporciona una referencia temporal adecuada para medir intervalos de ejecución en programas MPI.

Tened en cuenta que el tiempo total de una ejecución paralela debe considerar el tiempo necesario para completar el trabajo de todos los procesos, y no únicamente el tiempo consumido por uno de ellos.

f. Representación de la solución paralela

Elaborad un grafo que represente el funcionamiento de la solución paralela basada en MPI.

Identificad los procesos participantes, las tareas que realiza cada uno de ellos y las comunicaciones necesarias.

Representad los puntos de sincronización y, cuando corresponda, las dependencias existentes entre las diferentes fases de procesamiento.

g. Análisis y discusión de los resultados

En la memoria de la práctica, analizad los siguientes aspectos.

Sobre el código implementado

Comentad los fragmentos más relevantes de las versiones secuencial y paralela, prestando especial atención a:

- Los parámetros que determinan el tamaño del problema.
- Las estructuras de control que desempeñan un papel importante en el algoritmo.
- Las modificaciones realizadas para transformar la versión secuencial en una solución paralela, justificando las decisiones adoptadas.
- Las funciones MPI utilizadas, explicando su finalidad y su papel en la distribución del trabajo, la comunicación y la sincronización entre procesos.

Sobre el paralelismo explotado

Revisad los conceptos estudiados en la Unidad 3, «Computación paralela», e identificad:

- Los tipos de paralelismo que habéis aprovechado.
- El modelo de programación paralela utilizado.
- Los mecanismos de comunicación empleados, distinguiendo entre comunicaciones punto a punto y colectivas.
- El estilo de programación paralela adoptado.
- La estructura paralela resultante del programa.

Explicad también cómo habéis distribuido los datos y las tareas entre los procesos, y qué decisiones habéis tomado para reducir los costes de comunicación.

Sobre los resultados de las pruebas

Caracterizad la arquitectura del sistema utilizado para realizar las pruebas, incluyendo información como el número de procesadores y núcleos disponibles, los niveles de memoria caché y las características relevantes del sistema de memoria.

Responded de forma razonada a las siguientes cuestiones:

7. Calculad la ganancia en velocidad (speed-up) en función del número de procesos MPI y de los parámetros que determinan el tamaño del problema. Representad los resultados mediante gráficas que faciliten su interpretación.
8. Analizad cómo evoluciona el rendimiento al aumentar el número de procesos. ¿Se obtiene siempre una mejora? ¿Qué factores limitan la escalabilidad de la implementación?
9. Estudiad el impacto de las comunicaciones entre procesos. ¿Qué proporción del tiempo de ejecución se dedica al procesamiento y cuál a las comunicaciones y la sincronización? ¿Cómo influye este reparto en el rendimiento?

10. Comparad las distintas estrategias de paralelización implementadas. ¿Qué estrategia proporciona mejores resultados y en qué condiciones? Justificad vuestra respuesta.
11. ¿Cuál sería la ganancia máxima teórica? ¿Qué factores impiden alcanzarla en vuestra implementación?
12. Comparad la eficiencia de las implementaciones secuencial y paralela. ¿Cuál obtiene mejores resultados para los distintos tamaños del problema y configuraciones estudiadas?

Podéis utilizar gráficas bidimensionales o tridimensionales, según resulte más adecuado para representar y comparar los resultados.

Entregables de esta tarea

Los resultados de cada uno de los pasos anteriores ("Pasos recomendados"), así como el código modificado para su paralelización mediante MPI.

La entrega se realizará mediante **UACloud**, **antes de la sesión de prácticas de la semana del 16 de noviembre**. Dispondréis de **CINCO sesiones** para trabajar en esta práctica. La actividad de entrega indicará la fecha y hora exactas y los formatos admitidos.

Originalidad. *El trabajo que presentéis debe ser vuestro. Podéis utilizar código y otras fuentes externas si su uso está permitido, los citáis y explicáis qué habéis aportado o modificado. Las entregas se evaluarán conforme a las normas de integridad académica aplicables en la Universidad de Alicante.*